RAND SYSTEM

Robust Archival Navigation & Distribution

Complete Multi-Layer Design Specification
Formal Functions • Pseudo-Code in Python / JavaScript / SQL
Query Interpreters • Foundation-to-Sequence Workflows
For End-User Online Document Filing, Sorting & Distribution Platform

CORE CAPABILITIES

- Intelligent Metadata-Driven Filing Engine with AI Classification
- Multi-Criteria & ML-Powered Sorting with Custom Interpreters
- Secure Online Distribution: Expiring Links, RBAC, Audit Trails
- Full-Text + Metadata Search with Natural Language Query Interpreter
- Version Control, Hierarchical + Tag-Based Organization

1. FOUNDATION: Core Concepts of Document Filing & Sorting

Before building the RAND System, we establish the **foundation** — the fundamental principles that every layer, function, and interpreter will build upon. This ensures end-users understand *why* each component exists and how sequences of operations deliver reliable, scalable online document distribution.

1.1 What is Document Filing?

Filing is the structured storage of documents using **metadata** (data about data) rather than just physical location. In RAND, every document receives a rich metadata profile: title, author, category (hierarchical), tags (flat flexible), custom fields, OCR-extracted text, upload timestamp, version history, and AI-derived semantic vectors for intelligent retrieval.

1.2 What is Document Sorting?

Sorting goes beyond simple alphabetical or date order. RAND supports **multi-layer sorting interpreters** that combine deterministic rules (e.g., Category > Date > Title) with machine learning relevance scoring. This allows context-aware ordering: financial reports sorted by fiscal quarter then relevance to 'Q3 audit'.

1.3 Why Multi-Layer Architecture?

A single monolithic application fails at scale. RAND strictly separates concerns into **layers** with clear interfaces (contracts). Each layer has its own **interpreter** that translates high-level user intent into lower-layer actions. This enables independent scaling, testing, security auditing, and technology evolution.

Foundation Principle: Every higher layer is an *interpreter* of the layer below it. The Presentation Layer interprets user gestures into API calls. The Business Layer interprets those into data operations. The Data Layer interprets queries into storage primitives. This chain of interpreters creates traceable, auditable, and maintainable sequences.

2. MULTI-LAYER ARCHITECTURE OF RAND SYSTEM

The RAND System is architected in five primary layers plus cross-cutting concerns. Below is the visual blueprint followed by detailed layer descriptions and formal interfaces.

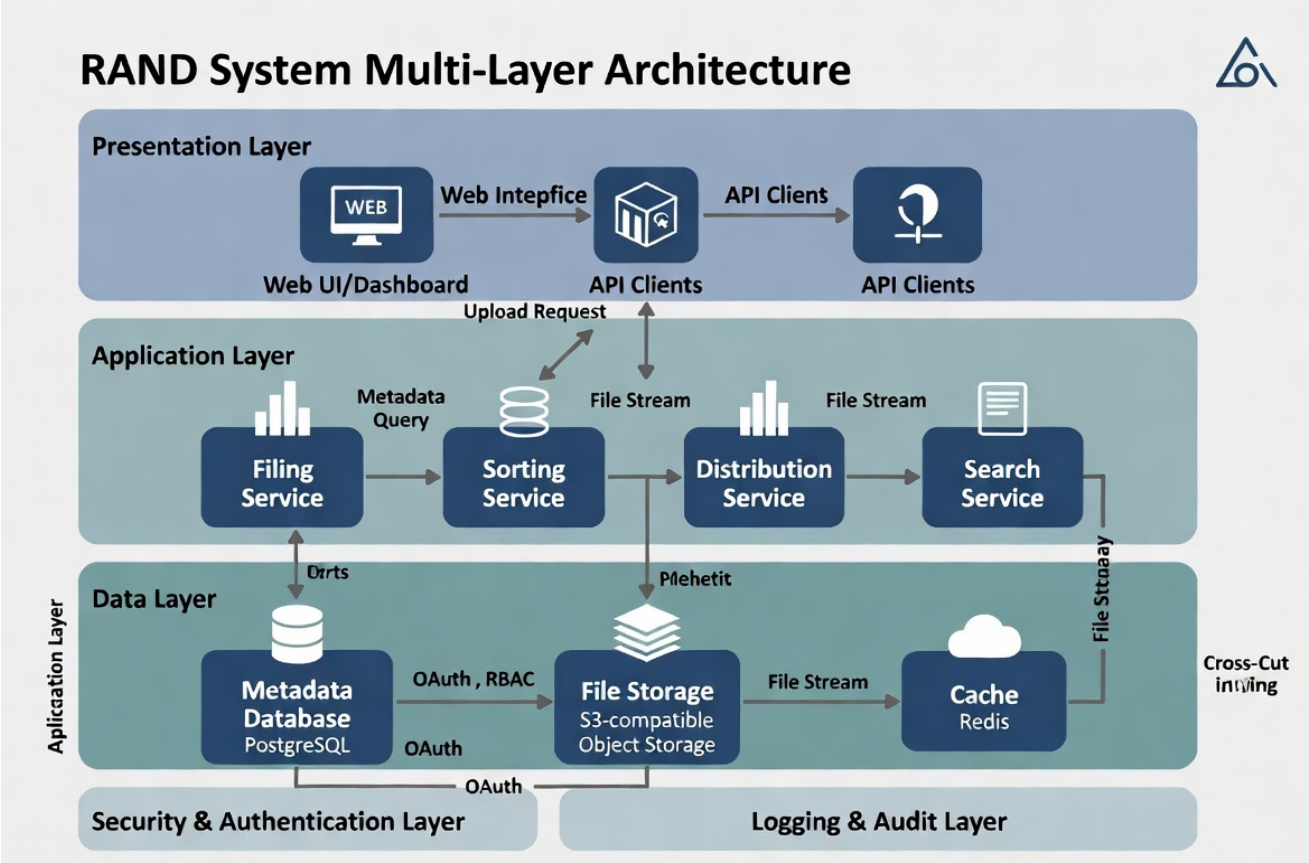

Figure 2.1: RAND Multi-Layer Architecture — Presentation, Application, Data, Security & Audit Layers

2.1 Presentation Layer (User-Facing Interpreter)

Interprets human actions (clicks, uploads, search queries) into structured API requests. Implemented as responsive React/Next.js dashboard + mobile PWA. Supports drag-and-drop filing, visual sort builders, and natural language search bar.

2.2 API Gateway & Orchestration Layer

Validates authentication (OAuth2/JWT + RBAC), rate limits, routes requests to microservices. Acts as the first formal interpreter: translates HTTP into internal command objects.

2.3 Application / Business Logic Layer

Core engines: **FilingEngine**, **SortingEngine**, **DistributionEngine**, **SearchEngine**. Each engine contains formal functions and sub-interpreters (e.g., SortInterpreter, QueryInterpreter). This is where business rules, AI classification, and workflow orchestration live.

2.4 Data Access & Storage Layer

Metadata in PostgreSQL (relational + JSONB for flexible fields). Binary files in S3-compatible object storage (MinIO / AWS S3). Full-text search via Elasticsearch or PostgreSQL tsvector. Caching hot metadata/queries in Redis.

2.5 Cross-Cutting: Security, Logging & Audit Layer

Every action is logged with who, what, when, why (purpose). RBAC + ABAC (Attribute-Based). Encryption at rest and in transit. Compliance hooks for GDPR, HIPAA, SOC2 via pluggable policy engines.

3. DATA MODEL — FOUNDATION FOR FILING & SORTING

The relational schema below powers all filing, sorting, and distribution logic. It is deliberately normalized for integrity yet flexible via JSONB metadata and many-to-many tag/category associations.

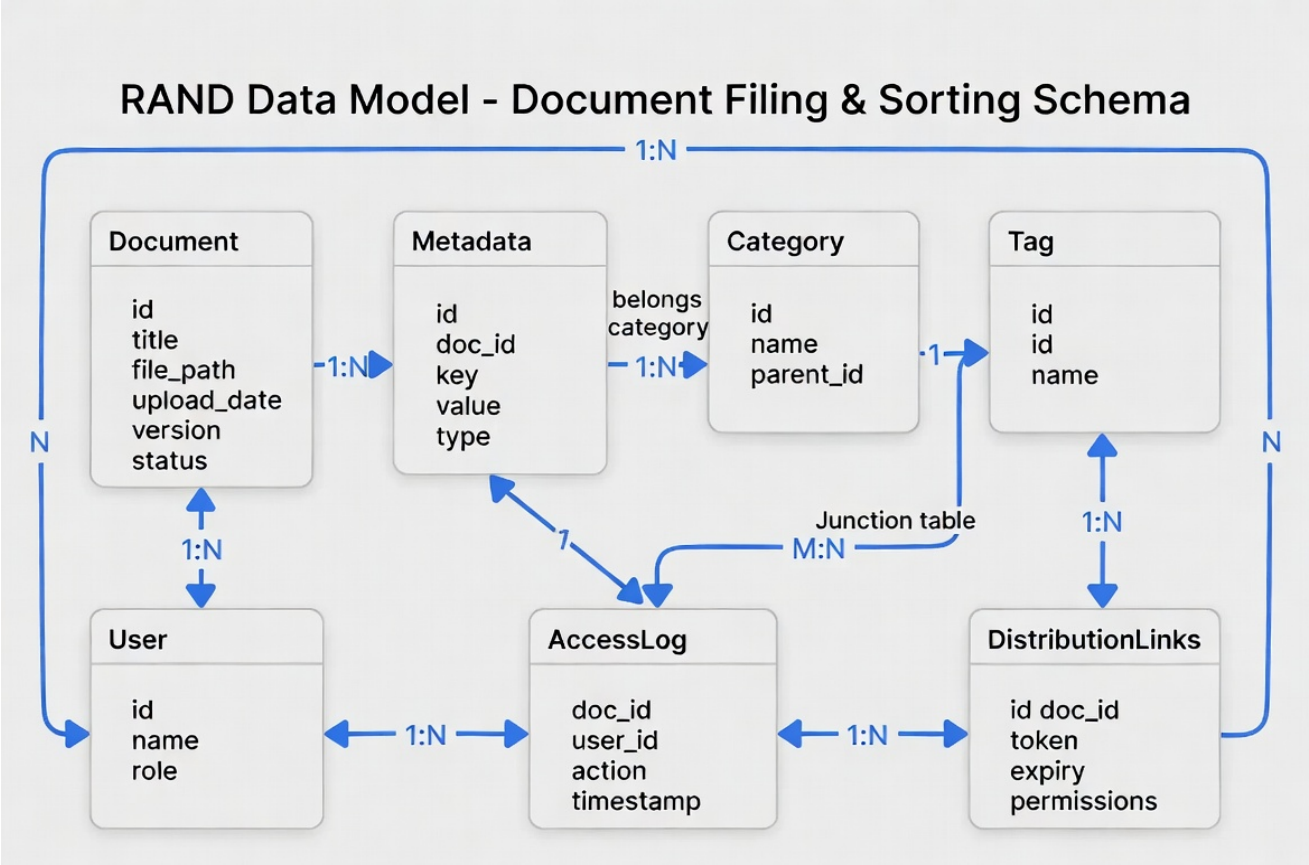

Figure 3.1: RAND Entity-Relationship Data Model — Core Entities for Document Filing, Sorting & Distribution

3.1 Key Tables & Their Roles in RAND Interpreters

Document — Central entity. Every filing operation creates/updates a Document row. Sorting engines join on this table.
Metadata — EAV (Entity-Attribute-Value) pattern for unlimited custom fields. QueryInterpreter dynamically builds WHERE clauses from this.
Category — Hierarchical (parent_id). FilingEngine uses tree traversal for folder-like navigation.
Tag — Many-to-many via junction. Enables powerful faceted filtering and ML tag prediction.
DistributionLinks — Powers the online distribution system. Each link has cryptographically secure token, expiry, and granular permissions.
AccessLog — Immutable audit trail. Every interpreter action appends here for compliance and analytics.

4. FORMAL FUNCTIONS — PRECISE SPECIFICATIONS

We now define the core operations using **formal function signatures** (mathematical style) followed by multi-language pseudo-code implementations. Each function includes an **interpreter explanation** — line-by-line commentary showing how the foundation concepts are realized in sequence.

4.1 file_document() — The Filing Engine Core

Formal Signature:
file_document(doc: DocumentBlob, meta: MetadataDict, user: UserContext) → DocumentID

Python Pseudo-Code (Business Logic Layer)

```
def file_document(doc_blob: bytes, metadata: dict, user: dict) -> str:
    # INTERPRETER STEP 1: Security & Validation Layer
    if not auth_service.has_permission(user, "document.create"):
        raise AuthorizationError("Insufficient privileges for filing")

    # INTERPRETER STEP 2: Data Layer - Persist binary
    file_id = object_storage.put_object(
        bucket="rand-documents",
        key=f"{user['org_id']}/{uuid4()}",
        data=doc_blob,
        metadata={"original_filename": metadata.get("filename")}
    )

    # INTERPRETER STEP 3: Metadata Layer + AI Classification
    enriched_meta = ai_classifier.enrich(metadata)  # OCR, NER, semantic tags
    doc_record = {
        "id": str(uuid4()),
        "file_path": file_id,
        "upload_date": datetime.utcnow(),
        "version": 1,
        "status": "active",
        **enriched_meta
    }
    doc_id = db.documents.insert(doc_record)

    # INTERPRETER STEP 4: Indexing for future Sorting/Search
    search_index.index_document(doc_id, enriched_meta)

    # INTERPRETER STEP 5: Audit
    audit_logger.log(user_id=user["id"], action="FILE", target=doc_id)

    return doc_id
```

Interpreter Note: Each step is a translation. Step 1 interprets user context into security decision. Step 3 interprets raw metadata into enriched, queryable form using AI. This creates the foundation for intelligent sorting later.

JavaScript Equivalent (Frontend Upload Handler)

```
async function fileDocument(file, metadata, userToken) {
  // Interpreter: FormData for multipart upload
  const formData = new FormData();
  formData.append('file', file);
  formData.append('metadata', JSON.stringify(metadata));

  const response = await fetch('/api/v1/documents', {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${userToken}` },
```

```
    body: formData
  });

  if (!response.ok) throw new Error('Filing failed at API Gateway');
  return response.json(); // { doc_id, status }
}
```

4.2 sort_documents() — The Sorting Engine with Multi-Layer Interpreter

Formal Signature:
sort_documents(docs: List[Document], criteria: List[SortCriterion], context: UserContext) → List[Document]
where SortCriterion = {field: string, direction: 'asc'|'desc', weight?: float, interpreter?: string}

The SortingEngine uses a **SortInterpreter** that can handle deterministic multi-key sorts, weighted scoring, and ML relevance reranking.

Python Pseudo-Code — SortInterpreter

```
class SortInterpreter:
    def __init__(self, criteria: list):
        self.criteria = criteria  # e.g. [{"field":"category", "dir":"asc"}, {"field":"date", "dir":"desc"}]

    def apply(self, documents: list, user_context: dict) -> list:
        # INTERPRETER: Build composite sort key function
        def composite_key(doc):
            keys = []
            for crit in self.criteria:
                val = self._get_field_value(doc, crit["field"])
                # Handle nested metadata
                if isinstance(val, str) and crit.get("interpreter") == "date":
                    val = datetime.fromisoformat(val)
                keys.append(val)
            return tuple(keys)

        # Primary sort (stable)
        sorted_docs = sorted(
            documents,
            key=composite_key,
            reverse=any(c["dir"] == "desc" for c in self.criteria)
        )

        # Optional ML rerank layer (second interpreter pass)
        if any(c.get("ml_model") for c in self.criteria):
            sorted_docs = self._ml_relevance_rerank(sorted_docs, user_context["query_intent"])

        return sorted_docs

    def _get_field_value(self, doc, field):
        # Interpreter for dynamic field access (supports dot notation & JSONB)
        if "." in field:
            base, sub = field.split(".", 1)
            return doc.get(base, {}).get(sub)
        return doc.get(field)
```

Interpreter Explanation: The SortInterpreter builds a dynamic key extractor at runtime. This allows end-users to define custom sort sequences via UI without code changes. The ML pass is an optional second interpreter that understands user intent beyond static fields.

SQL Equivalent (for simple deterministic sorts pushed to DB)

```
-- Interpreter translates UI sort builder into this dynamic query
SELECT d.*, m.value as custom_field
FROM documents d
LEFT JOIN metadata m ON d.id = m.doc_id AND m.key = 'priority'
WHERE d.status = 'active'
ORDER BY
    CASE WHEN :sort_field = 'category' THEN d.category END ASC,
    CASE WHEN :sort_field = 'upload_date' THEN d.upload_date END DESC,
```

```
    d.title ASC
LIMIT :page_size OFFSET :offset;
```

4.3 generate_distribution_link() — Online Distribution Engine

Formal Signature:
generate_distribution_link(doc_id: DocumentID, recipient: UserOrEmail, permissions: PermissionSet, expiry: Duration) → SecureLink

Python Pseudo-Code

```
def generate_distribution_link(doc_id: str, recipient: dict, permissions: list, expiry_hours: int = 72) -> dict:
    # INTERPRETER STEP 1: Validate recipient has base access
    if not distribution_service.can_share(doc_id, current_user):
        raise PermissionDenied("Document not shareable")

    # INTERPRETER STEP 2: Cryptographic token generation
    token = secrets.token_urlsafe(32)  # 256-bit secure random

    # INTERPRETER STEP 3: Persist link record (Data Layer)
    link_record = {
        "id": str(uuid4()),
        "doc_id": doc_id,
        "token": token,
        "created_by": current_user["id"],
        "recipient_email": recipient.get("email"),
        "permissions": permissions,  # e.g. ["view", "download", "comment"]
        "expires_at": datetime.utcnow() + timedelta(hours=expiry_hours),
        "access_count": 0
    }
    db.distribution_links.insert(link_record)

    # INTERPRETER STEP 4: Generate public URL
    share_url = f"https://rand.example.com/share/{token}"

    # INTERPRETER STEP 5: Notify & Audit
    if recipient.get("email"):
        email_service.send_share_notification(recipient["email"], share_url, doc_id)
    audit_logger.log(action="DISTRIBUTE", target=doc_id, details={"token": token[:8]+"..."})

    return {"url": share_url, "token": token, "expires_at": link_record["expires_at"]}
```

Sequence Interpreter: This function orchestrates Security → Crypto → Persistence → Notification → Audit. Each step can be independently scaled or replaced (e.g., swap email for Slack notification) without breaking the chain.

5. QUERY INTERPRETER — NATURAL LANGUAGE TO EXECUTION

One of RAND's most powerful features for end-users is the **QueryInterpreter**. It accepts natural language or structured queries and translates them into optimal execution plans across layers.

Example Query Flow (Foundation → Sequence)

User types: *"Show all finance reports from 2025, sorted by relevance to 'budget audit', filed under Q3, shared with me in last 30 days"*

```
# QueryInterpreter Pipeline (Python pseudo)
query = "finance reports 2025 budget audit Q3"
intent = nlp_parser.parse(query)  # → {"categories": ["finance"], "date_range": [2025-01-01,2025-12-31],
                                  #    "keywords": ["budget", "audit"], "sort": "relevance"}

# Layer translation sequence:
1. Security Interpreter → add user RBAC filters (only docs user can see)
2. Category Interpreter → resolve "Q3" to category tree node
3. Metadata Filter Interpreter → build WHERE on JSONB or EAV
4. Full-Text Interpreter → tsquery on indexed content + vector similarity
5. Sort Interpreter → apply relevance scoring (BM25 + embedding cosine)
6. Distribution Interpreter → join only docs with active DistributionLinks to current user
7. Pagination & Projection Interpreter → LIMIT + selected fields
```

6. END-TO-END SEQUENCE: UPLOAD → FILE → SORT → DISTRIBUTE

The following visual sequence diagram shows how all layers and interpreters work together in a complete user journey. This is the 'another sequence' built on the foundation — the full orchestration that delivers value to end-users.

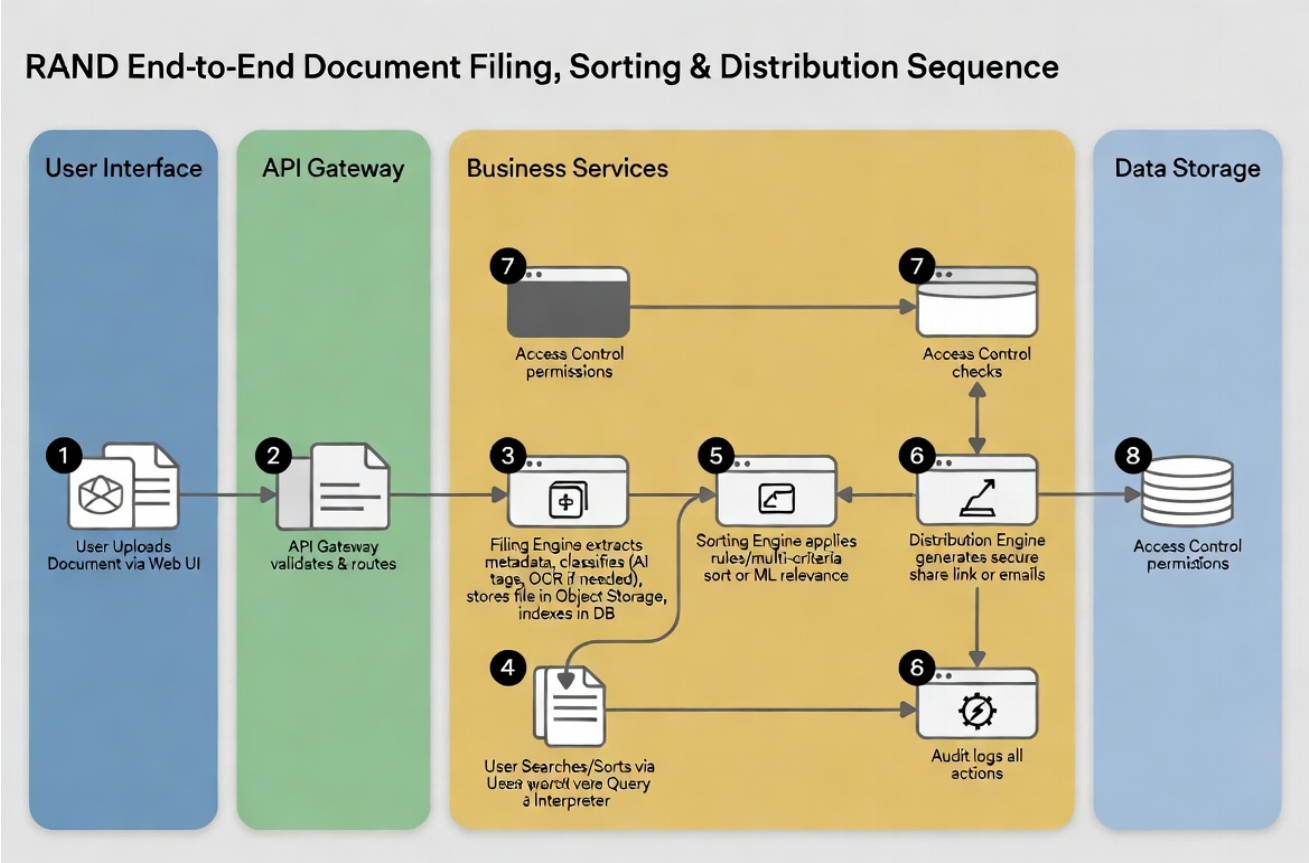

Figure 6.1: RAND Complete Lifecycle Sequence — From User Upload through Filing, Sorting, to Secure Online Distribution

6.1 Detailed Step-by-Step with Layer Transitions

Step 1-2 (Presentation → API Gateway)

User uploads via drag-drop. Frontend JS validates file type/size locally, then POSTs to /api/documents with JWT. Gateway verifies token, extracts org/user context, forwards to Filing Service.

Step 3 (Filing Engine)

Extracts MIME, runs OCR if PDF/image, runs NER/AI tagging, stores binary to Object Storage, writes metadata row, triggers indexing. All within a transactional boundary with rollback on failure.

Step 4 (Search Index Update)

Asynchronous but guaranteed via message queue. Enables instant searchability.

Step 5 (Sorting Engine - Optional)

If auto-sort rules configured for the category, documents are immediately placed in virtual sorted views.

Step 6-7 (Distribution)

User clicks 'Share' → DistributionEngine creates time-limited, permissioned link. Access control re-checked on every access (not just creation).

Step 8 (Audit)

Immutable log written. Can be queried for compliance reports or fed into analytics dashboards.

7. ONLINE DISTRIBUTION — API & CLIENT CODE PATTERNS

The distribution system must be secure, auditable, and easy for end-users. Below are production-ready patterns.

7.1 REST API Endpoints (FastAPI-style Pseudo)

```
# FastAPI pseudo-code for Distribution routes
from fastapi import APIRouter, Depends, HTTPException
router = APIRouter(prefix="/api/v1/distribution")

@router.post("/documents/{doc_id}/share")
async def create_share_link(
    doc_id: str,
    payload: ShareRequest,  # {recipient_email, permissions: list, expiry_hours}
    current_user = Depends(get_current_user)
):
    link = distribution_engine.generate_link(
        doc_id=doc_id,
        recipient=payload.recipient_email,
        permissions=payload.permissions,
        expiry_hours=payload.expiry_hours,
        created_by=current_user
    )
    return {"share_url": link.url, "expires": link.expires_at}

@router.get("/share/{token}")
async def access_shared_document(token: str, request: Request):
    link = db.get_link_by_token(token)
    if not link or link.expires_at < now():
        raise HTTPException(410, "Link expired or invalid")

    # Re-interpret permissions on every access
    if "view" not in link.permissions:
        raise HTTPException(403, "Insufficient permissions")

    # Stream file from object storage (no local copy)
    return StreamingResponse(
        object_storage.get_stream(link.doc.file_path),
        headers={"Content-Disposition": f"inline; filename={link.doc.title}"}
    )
```

7.2 Frontend React Hook for Distribution UI

```
// React hook example
function useDocumentDistribution(docId) {
  const [shareLink, setShareLink] = useState(null);

  const createLink = async (recipient, permissions, expiry) => {
    const res = await fetch(`/api/v1/distribution/documents/${docId}/share`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json', Authorization: `Bearer ${token}` },
      body: JSON.stringify({ recipient_email: recipient, permissions, expiry_hours: expiry })
    });
    const data = await res.json();
    setShareLink(data);
    return data;
  };

  return { createLink, shareLink };
}
```

8. IMPLEMENTATION ROADMAP & TECH STACK

To move from this design document to a running online system, follow the foundation-to-sequence path:

Phase 1 — Foundation (Weeks 1-4): PostgreSQL schema + Object Storage (MinIO) + basic file_document() function. Deploy behind API Gateway (Kong or AWS API Gateway).

Phase 2 — Interpreters (Weeks 5-8): Implement SortInterpreter + QueryInterpreter. Add Elasticsearch for full-text. Build simple React dashboard for upload + search.

Phase 3 — Distribution & Security (Weeks 9-12): DistributionEngine + RBAC + audit logging. Add expiring links and email notifications.

Phase 4 — Intelligence & Polish (Weeks 13-16): AI classification pipeline (spaCy / HuggingFace), ML reranking in SortInterpreter, advanced analytics dashboard on AccessLogs.

Recommended Stack:
• Backend: Python/FastAPI or Node.js/NestJS (microservices)
• Frontend: React + TypeScript + Tailwind + shadcn/ui
• Database: PostgreSQL 16+ (JSONB, full-text, row-level security)
• Storage: MinIO or AWS S3
• Search: Elasticsearch or Typesense
• Auth: Auth0 / Keycloak / Ory
• AI: Hugging Face Transformers for classification & embeddings
• Orchestration: Docker + Kubernetes or Railway/Render for simpler deploys

RAND SYSTEM — KEY TAKEAWAYS FOR END-USERS

✓ Multi-layer design ensures security, scalability, and maintainability

✓ Formal functions + interpreters make every operation traceable and customizable

✓ Foundation concepts (metadata, indexing) power advanced sequences (AI sort, secure share)

✓ Online distribution is first-class: every document can be safely shared with granular control

✓ Complete auditability built into every layer transition

This document provides the complete blueprint. Developers can implement layer-by-layer using the formal signatures and pseudo-code as executable specifications. End-users benefit from an intuitive, powerful platform where filing feels natural, sorting is intelligent, and distribution is secure by design.

— End of RAND System Design Specification —

Built with formal rigor, multi-layer interpreters, and end-user sequences in mind. | 2026